

The gRPC-based client

Ideally, you will use the stub that's generated by the server or use the same service .proto file to generate one in the client for calling the stub from the client code. However, we'll add `grpc-server-service` as a dependency in the `pom.xml` file of `grpc-client-service`. This is shown in the following example:

```
<dependency>
  <groupId>${project.groupId}</groupId>
  <artifactId>grpc-server-service</artifactId>
  <version>${project.version}</version>
</dependency>
```

That's how we can simply write the method that will call the server and get the response. The `io.grpc.ManagedChannelBuilder` class is used for creating the channel (`io.grpc.ManagedChannel`) to the server. This channel is then passed to the auto generated `EmployeeService` stub (`EmployeeServiceGrpc.EmployeeServiceBlockingStub`). Then, you can simply use this stub to make the server calls. You can close the server connection by calling the `channel.shutdown()` method, as shown in the following example:

```
public class GrpcClient {

    public static void main(String[] args) throws InterruptedException {
        ManagedChannel channel =
            ManagedChannelBuilder.forAddress("localhost", 8080)
                .usePlaintext()
                .build();

        EmployeeServiceGrpc.EmployeeServiceBlockingStub stub =
            EmployeeServiceGrpc.newBlockingStub(channel);

        Response createResponse = stub.create(
            Employee.newBuilder().setFirstName("Mark")
                .setLastName("Butcher")
                .setDeptId(1L).setSalary(200000)
        );
    }
}
```

```

        .setAddress(Employee.Address.newBuilder()
        .setHouseNo("406").setStreet("Hill View")
            .setCity("San Jose")
            .setState("State")
            .setCountry("US").build())
        .build());
System.out.println("Create Response: \n" + createResponse);

Response updateResponse = stub.update(
    Employee.newBuilder().setId(2)
        .setFirstName("Mark").setLastName("Butcher II")
        .setDeptId(1L).setSalary(200000)
        .setAddress(Employee.Address.newBuilder()
        .setHouseNo("406").setStreet("Mountain View")
            .setCity("San Jose")
            .setState("State")
            .setCountry("US").build())
        .build());
System.out.println("Update Response: \n" + updateResponse);

EmployeeList employeeList =
    stub.list(com.packtpub.mmj.grpc.server.Empty.newBuilder()
        .build());
System.out.println("Employees: \n" + employeeList);

Response deleteResponse = stub.delete(EmployeeId.newBuilder()
    .setId(2).build());
System.out.println("Delete Response: \n" + deleteResponse);
channel.shutdown();
    }
}

```

You can directly run the client from the IDE, or by building the fat jar and then executing it with the `Java -jar <jar>` command. The output of the gRPC client is as follows:

```

Create Response:
code: 201
message: "CREATED"

```

```

Update Response:
code: 200
message: "UPDATED"

```

```
Employees:
employees {
  id: 1
  firstName: "John"
  lastName: "Mathews"
  deptId: 1
  salary: 100000.0
  address {
    houseNo: "604"
    street: "Park View"
    city: "Atlanta"
    state: "State"
    country: "US"
  }
}
employees {
  id: 2
  firstName: "Mark"
  lastName: "Butcher II"
  deptId: 1
  salary: 200000.0
  address {
    houseNo: "406"
    street: "Mountain View"
    city: "San Jose"
    state: "State"
    country: "US"
  }
}

Delete Response:
code: 200
message: "DELETED"
```

We have used the Protobuf for serialization. If you are interested in using the JSON, then you will need to have a look at <https://grpc.io/blog/grpc-with-json>.

